- **الاسم** : ريم عمر السيد محمد السرنجاوي

- **الرقم الجامعي** : 2501760

- **عنوان البحث** : JavaScript

المقدمة وأساسيات تفاعل واجهات الويب

1. مقدمة البحث (Introduction)

تطورت شبكة الويب العالمية (WWW) بشكل مذهل خلال العقود الأخيرة؛ حيث تحولت المواقع من صفحات نصية ثابتة (Static Pages) تعتمد فقط على HTML و CSS، إلى تطبيقات ويب تفاعلية وديناميكية (Dynamic Web Applications). العنصر الأساسي والمحرك الرئيسي لهذا التحول هو لغة JavaScript.

تعتبر لغة JavaScript لغة برمجة قائمة على الأحداث (Event-Driven Programming Language). هذا يعني أن البرمجية لا تنتظر تنفيذاً خطياً متسلسلاً فحسب، بل تقبع في حالة "استماع" دائم لما يفعله المستخدم (مثل ضغطات الفأرة، الكتابة على لوحة المفاتيح، أو تحريك الشاشة). يهدف هذا البحث إلى دراسة الكيفية التي تترجم بها المتصفحات ضغطات المستخدم إلى استجابات فورية وتفاعلية.

2. نموذج كائنات المستند (DOM) وعلاقته بالتفاعل

قبل فهم كيفية التعامل مع الضغطات، يجب فهم البيئة التي تعمل فيها JavaScript داخل المتصفح، وهي DOM (Document Object Model).

* تعريف الـ DOM: هو تمثيل هيكلي على شكل شجرة (Tree Structure) لكل عناصر صفحة الويب (مثل الأزرار، العناوين، الفقرات).

* دور JavaScript: تقوم اللغات البرمجية بالوصول إلى هذه الشجرة وتعديل خصائصها. عندما يضغط المستخدم على زر معين، تدرك JavaScript العنصر المستهدف بدقة داخل شجرة الـ DOM وتقوم بتحديث الصفحة دون الحاجة لإعادة تحميلها بالكامل (Reload).

3. مفهوم الأحداث (What is an Event?)

في عالم تطوير الويب، يُعرف الحدث (Event) بأنه إشارة (Signal) يرسلها المتصفح لتوضيح أن شيئاً ما قد حدث نتيجة تفاعل المستخدم أو النظام.

أشهر أنواع الأحداث المتعلقة بضغطات المستخدم:

* click: يتم إطلاقه عندما يضغط المستخدم بزر الفأرة الأيسر على عنصر ما (أو يضغط بإصبعه على الشاشات اللمسية).

* dblclick: يتم إطلاقه عند الضغط المزدوج (مرتين متتاليتين).

* mousedown & mouseup: يتم إطلاق الأول بمجرد نزول زر الفأرة لأسفل، والثاني عند تحرير الزر ليرتفع لأعلى.

* contextmenu: يتم إطلاقه عند الضغط بزر الفأرة الأيسر/الأيمن لفتح القائمة المختصرة.

pagebreak\

آليات الاستماع للأحداث (Event Listeners)

4. كيف تستمع JavaScript لضغطات المستخدم؟

تتيح JavaScript ثلاث طرق رئيسية لربط ضغطة المستخدم بالكود البرمجي (وظيفة الاستجابة):

أولاً: معالجات الأحداث المضمنة (Inline Event Handlers)

تتم كتابة كود الجافا سكريبت مباشرة داخل وسم الـ HTML.

* مثال:html

<button onclick="alert('تم الضغط!')">اضغط هنا</button>

يُرجى استخدام الرمز البرمجي بحذر.

* عيوبها: تفصل كود البرمجة عن التصميم، وتجعل الصيانة صعبة جداً في المشاريع الكبيرة.

ثانياً: خصائص عناصر DOM (DOM Object Properties)

يتم اختيار العنصر في ملف الجافا سكريبت ثم إسناد دالة للحدث الخاص به.

* مثال:javascript

```
const myButton = document.getElementById("btn");

myButton.onclick = function() {
    console.log("تمت الضغطة بنجاح");
};
```

يُرجى استخدام الرمز البرمجي بحذر.

* عيوبها: لا تسمح بإضافة أكثر من دالة (Listener) لنفس الحدث على نفس العنصر؛ فالتابع الجديد يلغي القديم.

ثالثاً: الطريقة الحديثة والموصى بها (addEventListener)

هي الطريقة القياسية والأكثر مرونة، حيث تتيح إضافة عدة وظائف لنفس الحدث دون تداخل.

* الصيغة العامة:javascript

```
element.addEventListener('event', function/callback);
```

يُرجى استخدام الرمز البرمجي بحذر.

* مثال تطبيقي:javascript

```
const saveButton = document.querySelector(".save-btn");

saveButton.addEventListener("click", () => {
    alert("تم حفظ البيانات!");
});
```

});

يُرجى استخدام الرمز البرمجي بحذر.

5. كائن الحدث (The Event Object - e)

عندما تقع ضغطة من المستخدم، يقوم المتصفح تلقائياً بإنشاء كائن يحتوي على كافة التفاصيل المتعلقة بهذه الضغطة ويمرره كمعامل (Argument) للدالة المستدعاة. يحتوي هذا الكائن على معلومات حيوية مثل:

* e.target: يحدد بدقة العنصر الذي ضغط عليه المستخدم.

* e.type: نوع الحدث (مثل "click").

* e.clientX و e.clientY: الإحداثيات الجغرافية (بالمستطيل المعتمد على البكسل) لمكان الضغطة على الشاشة.

دورة حياة الحدث (Event Lifecycle) والتطبيقات المتقدمة

6. دورة حياة الحدث: التدفق والانتشار (Event Propagation)

عندما يضغط المستخدم على زر يقع داخل قائمة، والقائمة تقع داخل صندوق (Div)، فإن الضغطة لا تؤثر على الزر وحده بل تنتقل عبر شجرة الـ DOM. تمر هذه العملية بثلاث مراحل رئيسية:

[1] مرحلة الالتقاط (Capturing Phase) -> ينتقل الحدث من أعلى الشجرة (Window) نزولاً للعنصر.

[2] مرحلة الهدف (Target Phase) -> يصل الحدث للعنصر الذي تم الضغط عليه بالفعل.

[3] مرحلة الفقاعة (Bubbling Phase) -> يصعد الحدث من العنصر المستهدف إلى أعلى الشجرة مجدداً.

• **ظاهرة الصعود (Event Bubbling):**

هي السلوك الافتراضي في المتصفحات؛ فإذا ضغطت على زر، يُعتبر أنك ضغطت أيضاً على الحاوية (Parent) التي تحويه.

• كيفية إيقاف هذا السلوك: إذا أراد المطور منع الحدث من الصعود وتنشيط عناصر أخرى، يستخدم الدالة المدمجة:javascript

e.stopPropagation();

يُرجى استخدام الرمز البرمجي بحذر.

• **منع السلوك الافتراضي (preventDefault):**

تمتلك بعض العناصر سلوكاً تلقائياً في المتصفح (مثل الرابط <a> الذي ينقل لصفحة أخرى عند الضغط عليه، أو أزرار الإرسال في النماذج Form التي تعيد تحميل الصفحة). يمكن لـ JavaScript إلغاء هذا السلوك عبر:

javascript

;()e.preventDefault

يُرجى استخدام الرمز البرمجي بحذر.

7. تفويض الأحداث (Event Delegation)

هو أسلوب متقدم وذكي لتحسين أداء المواقع. بدلاً من وضع مستمع أحداث (addEventListener) على 100 زر داخل قائمة، يتم وضع مستمع واحد فقط على العنصر الأب (Parent)، وبفضل خاصية الـ Bubbling وكائن الحدث e.target يمكن معرفة أي زر تم ضغطه تحديداً وتعديله. هذا يوفر استهلاك ذاكرة الجهاز (RAM) بشكل كبير جداً.

الخلاصة والمراجع

8. الخاتمة (Conclusion)

تُظهر دراسة آليات التعامل مع الأحداث في JavaScript مدى الذكاء والهيكلية التي بُنيت عليها واجهات الويب الحديثة. إن فهم كيفية التقاط ضغطات المستخدم، ومعالجة كائن الحدث (Event Object)، وإدارة تدفق الأحداث عبر شجرة الـ DOM ليس مجرد رفاهية برمجية، بل هو الركيزة الأساسية لبناء مواقع سريعة الاستجابة، خفيفة على أجهزة المستخدمين، وتوفر تجربة مستخدم (UX) ممتازة وسلسة. بفضل هذه التقنيات، تحولت المتصفحات من مجرد قوارئ للمستندات إلى بيئات تشغيل متكاملة لأعقد التطبيقات البرمجية.

9. المراجع (References)

1. Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.

2. MDN Web Docs (Mozilla Developer Network). Introduction to events. المتوفر على الرابط الرسمي لتوثيقات موزيلا.

3. W3Schools Online Web Tutorials. JavaScript HTML DOM EventListener.

4. Haverbeke, M. (2018). Eloquent JavaScript: A Modern Introduction to Programming (3rd ed.). No Starch Press.

5. Javascript.info. The Desktop App & Browser Events: Bubbling and capturing